WIRESHARK is a tool used to monitor and analyze network traffic.

Documentation

Virtual Machines on Virtualbox
- Windows 11 VM (Wireshark workstation)
- Ubuntu Linux server
- windows server 2022
- Kali Linux

Tools:
- Wireshark
- Command Prompt
- Ping
- SSH

STEP 1

Kali IP: 192.168.56.103
Ubuntu IP: 192.168.56.102

Ping test = success

Install wireshark on kali

```
Sudo apt update
Sudo apt install wireshark -y
```

After installing wireshark neither eth0 or eth1 show traffic lets check id wireshark has permission to capture packets

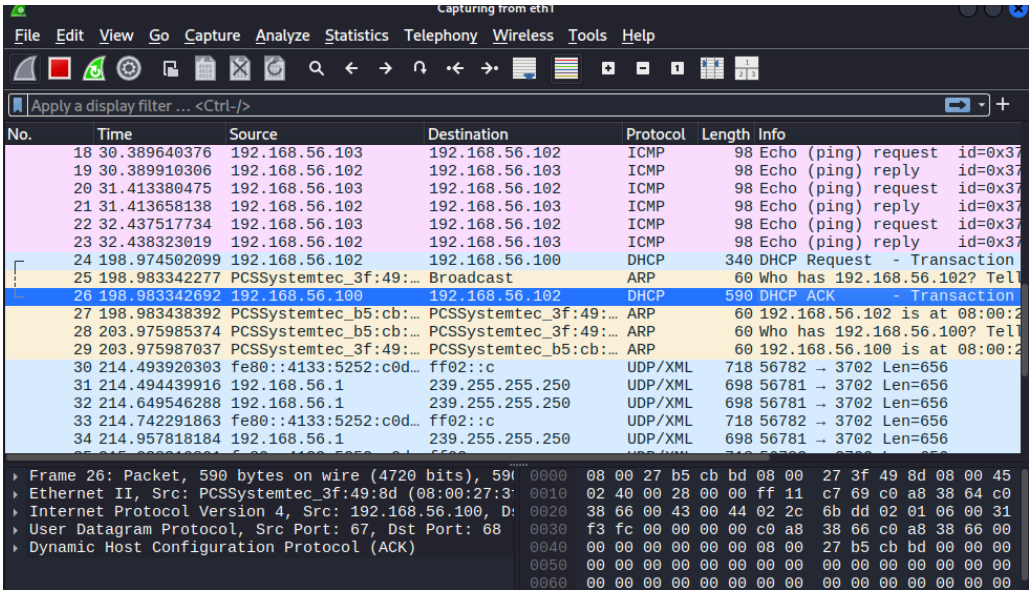
- Lets check what interface kali has
- Looks like kali has 2 interfaces both eth0 and eth1 (I used NAT and host-only/internal network)

Force traffic on eth1 by double clicking
Tested pinging Ubuntu on Kali command and was able to see ICMP echo request and echo reply, ARP broadcast, and DHCP ACK captures

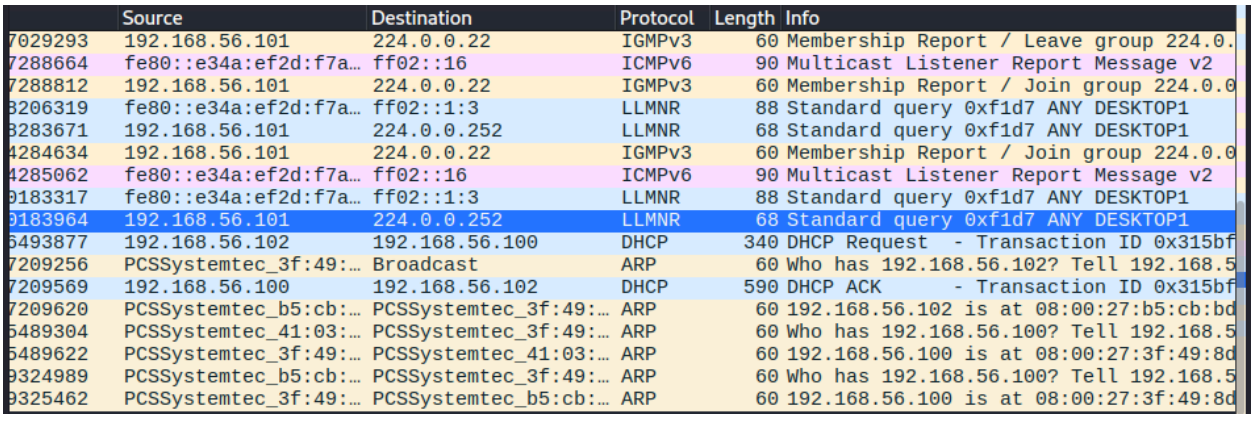
Analyze an Echo request packet
Source: 192.168.56.103 Kali

Destination: 192.168.56.102 Ubuntu
Type: Echo (ping) request (8)
Code: 0

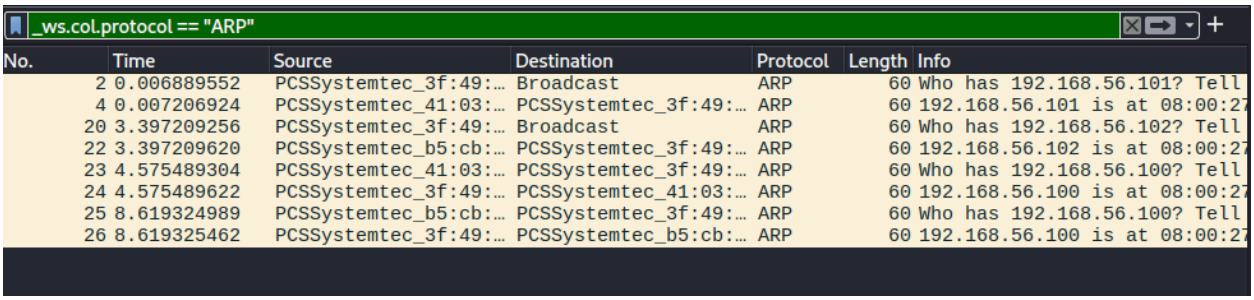
Different types and codes mean
Type 8 = request
Type 0 = reply
Code 0 = normal ping



LLMNR = windows machine use it for local name resolution
The LLMNR captures show that kali and the window machine are already communicating. I want to make sure of the Ip address of the windows 11 (192.168.56.101)



Applying a filter
ARP



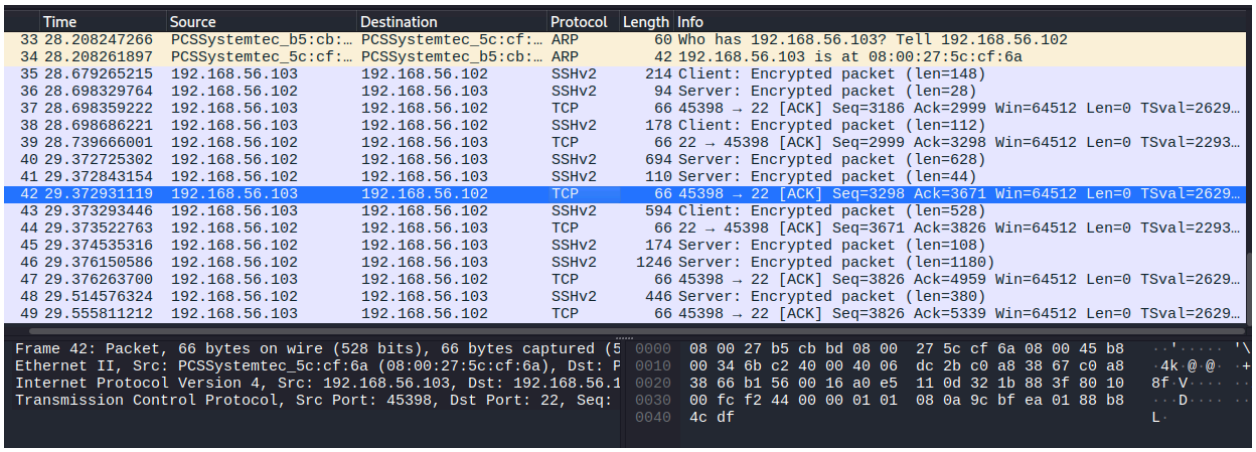
_ws.col.protocol == "ARP"

No.	Time	Source	Destination	Protocol	Length	Info
2	0.006889552	PCSSystemtec_3f:49:…	Broadcast	ARP	60	Who has 192.168.56.101? Tell
4	0.007206924	PCSSystemtec_41:03:…	PCSSystemtec_3f:49:…	ARP	60	192.168.56.101 is at 08:00:27
20	3.397209256	PCSSystemtec_3f:49:…	Broadcast	ARP	60	Who has 192.168.56.102? Tell
22	3.397209620	PCSSystemtec_b5:cb:…	PCSSystemtec_3f:49:…	ARP	60	192.168.56.102 is at 08:00:27
23	4.575489304	PCSSystemtec_41:03:…	PCSSystemtec_3f:49:…	ARP	60	Who has 192.168.56.100? Tell
24	4.575489622	PCSSystemtec_3f:49:…	PCSSystemtec_41:03:…	ARP	60	192.168.56.100 is at 08:00:27
25	8.619324989	PCSSystemtec_b5:cb:…	PCSSystemtec_3f:49:…	ARP	60	Who has 192.168.56.100? Tell
26	8.619325462	PCSSystemtec_3f:49:…	PCSSystemtec_b5:cb:…	ARP	60	192.168.56.100 is at 08:00:27

I did a **nmap** scan for open ports on the Ubuntu server.
Results: 999 closed tcp ports
22/ Tcp open ssh

```
┌──(kali㉿kali)-[~]
└─$ nmap -A 192.168.56.102
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-01 11:52 -0400
Nmap scan report for 192.168.56.102
Host is up (0.00044s latency).
Not shown: 999 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 10.2p1 Ubuntu 2ubuntu3.2 (Ubuntu Linux; protocol 2.0)
MAC Address: 08:00:27:B5:CB:BD (Oracle VirtualBox virtual NIC)
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
TCP/IP fingerprint:
OS:SCAN(V=7.98%E=4%D=6/1%OT=22%CT=1%CU=41169%PV=Y%DS=1%DC=D%G=Y%M=080027%TM
OS:=6A1DAAE7%P=x86_64-pc-linux-gnu)SEQ(SP=102%GCD=1%ISR=107%TI=Z%CI=Z%TS=21
OS:)SEQ(SP=102%GCD=2%ISR=10D%TI=Z%CI=Z%II=I%TS=21)SEQ(SP=104%GCD=1%ISR=106%
OS:TI=Z%CI=Z%II=I%TS=22)SEQ(SP=104%GCD=1%ISR=10B%TI=Z%CI=Z%II=I%TS=21)SEQ(S
OS:P=FD%GCD=1%ISR=106%TI=Z%CI=Z%II=I%TS=22)OPS(O1=M5B4ST11NWA%O2=M5B4ST11NW
OS:A%O3=M5B4NNT11NWA%O4=M5B4ST11NWA%O5=M5B4ST11NWA%O6=M5B4ST11)WIN(W1=FE88%
OS:W2=FE88%W3=FE88%W4=FE88%W5=FE88%W6=FE88)ECN(R=Y%DF=Y%T=40%W=FAF0%O=M5B4N
OS:NSNWA%CC=Y%Q=)T1(R=Y%DF=Y%T=40%S=O%A=S+%F=AS%RD=0%Q=)T2(R=N)T3(R=N)T4(R=
OS:Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T5(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=A
OS:R%O=%RD=0%Q=)T6(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T7(R=Y%DF=Y%T=4
OS:0%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=
OS:G%RIPCK=G%RUCK=G%RUD=G)IE(R=Y%DFI=N%T=40%CD=S)
```

Wireshark can see the network conversation, it cannot read your password or commands because SSH encrypts everything.
It will show the 3-way handshake.



	Time	Source	Destination	Protocol	Length	Info
33	28.208247266	PCSSystemtec_b5:cb:…	PCSSystemtec_5c:cf:…	ARP	60	Who has 192.168.56.103? Tell 192.168.56.102
34	28.208261897	PCSSystemtec_5c:cf:…	PCSSystemtec_b5:cb:…	ARP	42	192.168.56.103 is at 08:00:27:5c:cf:6a
35	28.679265215	192.168.56.103	192.168.56.102	SSHv2	214	Client: Encrypted packet (len=148)
36	28.698329764	192.168.56.102	192.168.56.103	SSHv2	94	Server: Encrypted packet (len=28)
37	28.698359222	192.168.56.103	192.168.56.102	TCP	66	45398 → 22 [ACK] Seq=3186 Ack=2999 Win=64512 Len=0 TSval=2629…
38	28.698686221	192.168.56.103	192.168.56.102	SSHv2	178	Client: Encrypted packet (len=112)
39	28.739666001	192.168.56.102	192.168.56.103	TCP	66	22 → 45398 [ACK] Seq=2999 Ack=3298 Win=64512 Len=0 TSval=2293…
40	29.372725302	192.168.56.102	192.168.56.103	SSHv2	694	Server: Encrypted packet (len=628)
41	29.372843154	192.168.56.102	192.168.56.103	SSHv2	110	Server: Encrypted packet (len=44)
42	29.372931119	192.168.56.103	192.168.56.102	TCP	66	45398 → 22 [ACK] Seq=3298 Ack=3671 Win=64512 Len=0 TSval=2629…
43	29.373293446	192.168.56.103	192.168.56.102	SSHv2	594	Client: Encrypted packet (len=528)
44	29.373522763	192.168.56.102	192.168.56.103	TCP	66	22 → 45398 [ACK] Seq=3671 Ack=3826 Win=64512 Len=0 TSval=2293…
45	29.374535316	192.168.56.102	192.168.56.103	SSHv2	174	Server: Encrypted packet (len=108)
46	29.376150586	192.168.56.102	192.168.56.103	SSHv2	1246	Server: Encrypted packet (len=1180)
47	29.376263700	192.168.56.103	192.168.56.102	TCP	66	45398 → 22 [ACK] Seq=3826 Ack=4959 Win=64512 Len=0 TSval=2629…
48	29.514576324	192.168.56.102	192.168.56.103	SSHv2	446	Server: Encrypted packet (len=380)
49	29.555811212	192.168.56.103	192.168.56.102	TCP	66	45398 → 22 [ACK] Seq=3826 Ack=5339 Win=64512 Len=0 TSval=2629…

```
Frame 42: Packet, 66 bytes on wire (528 bits), 66 bytes captured (5
Ethernet II, Src: PCSSystemtec_5c:cf:6a (08:00:27:5c:cf:6a), Dst: P
Internet Protocol Version 4, Src: 192.168.56.103, Dst: 192.168.56.1
Transmission Control Protocol, Src Port: 45398, Dst Port: 22, Seq:
```

```
0000  08 00 27 b5 cb bd 08 00  27 5c cf 6a 08 00 45 b8   ··'····· '\·j··E·
0010  00 34 6b c2 40 00 40 06  dc 2b c0 a8 38 67 c0 a8   ·4k·@·@· ·+··8g··
0020  38 66 b1 56 00 16 a0 e5  11 0d 32 1b 88 3f 80 10   8f·V···· ··2··?··
0030  00 fc f2 44 00 00 01 01  08 0a 9c bf ea 01 88 b8   ···D···· ········
0040  4c df                                              L·
```

DNS

Looking for domains in wireshark

3-WAY handshake

[SYN] client sends to server

[SYN, ACK] server sends back that it has received and acknowledged.

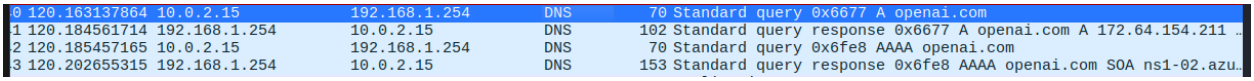
[ACK] client acknowledges as well.

[PSH] DATA

[FIN] connection is closed

To capture DNS traffic

I did nslookup openai.com or pinging openai.com

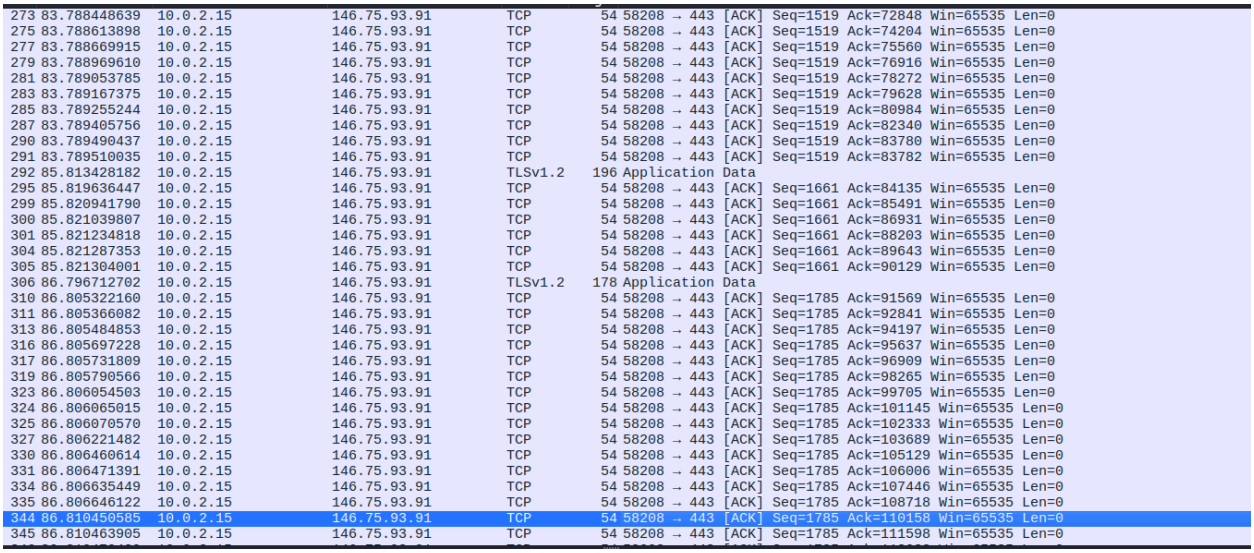


```
0 120.163137864 10.0.2.15       192.168.1.254   DNS      70 Standard query 0x6677 A openai.com
1 120.184561714 192.168.1.254   10.0.2.15       DNS     102 Standard query response 0x6677 A openai.com A 172.64.154.211 …
2 120.185457165 10.0.2.15       192.168.1.254   DNS      70 Standard query 0x6fe8 AAAA openai.com
3 120.202655315 192.168.1.254   10.0.2.15       DNS     153 Standard query response 0x6fe8 AAAA openai.com SOA ns1-02.azu…
```

I look like all these captures belong to RIPE network coordination Centre (large hosting/CDN network)

Notice the [ACK]

- Vm INITIATED THE CONNECTION
- Port 443 = HTTPS
- Data is encrypted
- This is normal web traffic

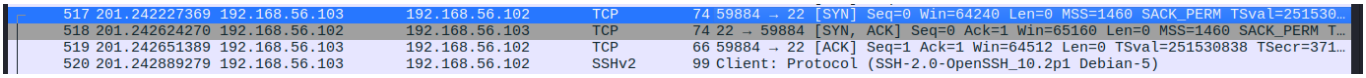


```
273 83.788448639  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1519 Ack=72848 Win=65535 Len=0
275 83.788613898  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1519 Ack=74204 Win=65535 Len=0
277 83.788669915  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1519 Ack=75560 Win=65535 Len=0
279 83.788969610  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1519 Ack=76916 Win=65535 Len=0
281 83.789053785  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1519 Ack=78272 Win=65535 Len=0
283 83.789167375  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1519 Ack=79628 Win=65535 Len=0
285 83.789255244  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1519 Ack=80984 Win=65535 Len=0
287 83.789405756  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1519 Ack=82340 Win=65535 Len=0
290 83.789490437  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1519 Ack=83780 Win=65535 Len=0
291 83.789510035  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1519 Ack=83782 Win=65535 Len=0
292 85.813428182  10.0.2.15       146.75.93.91    TLSv1.2 196 Application Data
295 85.819636447  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1661 Ack=84135 Win=65535 Len=0
299 85.820941790  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1661 Ack=85491 Win=65535 Len=0
300 85.821039807  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1661 Ack=86931 Win=65535 Len=0
301 85.821234818  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1661 Ack=88203 Win=65535 Len=0
304 85.821287353  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1661 Ack=89643 Win=65535 Len=0
305 85.821304001  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1661 Ack=90129 Win=65535 Len=0
306 86.796712702  10.0.2.15       146.75.93.91    TLSv1.2 178 Application Data
310 86.805322160  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1785 Ack=91569 Win=65535 Len=0
311 86.805366082  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1785 Ack=92841 Win=65535 Len=0
313 86.805484853  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1785 Ack=94197 Win=65535 Len=0
316 86.805697228  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1785 Ack=95637 Win=65535 Len=0
317 86.805731809  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1785 Ack=96909 Win=65535 Len=0
319 86.805790566  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1785 Ack=98265 Win=65535 Len=0
323 86.806054503  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1785 Ack=99705 Win=65535 Len=0
324 86.806065015  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1785 Ack=101145 Win=65535 Len=0
325 86.806070570  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1785 Ack=102333 Win=65535 Len=0
327 86.806221482  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1785 Ack=103689 Win=65535 Len=0
330 86.806460614  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1785 Ack=105129 Win=65535 Len=0
331 86.806471391  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1785 Ack=106006 Win=65535 Len=0
334 86.806635449  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1785 Ack=107446 Win=65535 Len=0
335 86.806646122  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1785 Ack=108718 Win=65535 Len=0
344 86.810450585  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1785 Ack=110158 Win=65535 Len=0
345 86.810463905  10.0.2.15       146.75.93.91    TCP      54 58208 → 443 [ACK] Seq=1785 Ack=111598 Win=65535 Len=0
```

How would an attack look like

- Many [SYN] packets
- Many destination ports
- Many destination IPs.
- Attacker could try to reach (22, 80, 443, 3349, 445)

SSH session showing the 3-way handshake between client and server.



This is also helpful to see if the connection succeeded

If missing the 3-way handshake

- Firewall issue
- Host down
- Packet drop

Finding open ports

On KALI

```
Nmap -sS 192.168.56.102
```

```
This doesn't show the 3-way handshake, this scans for open ports, by checking each port.
```

56075 → 4443 [SYN]

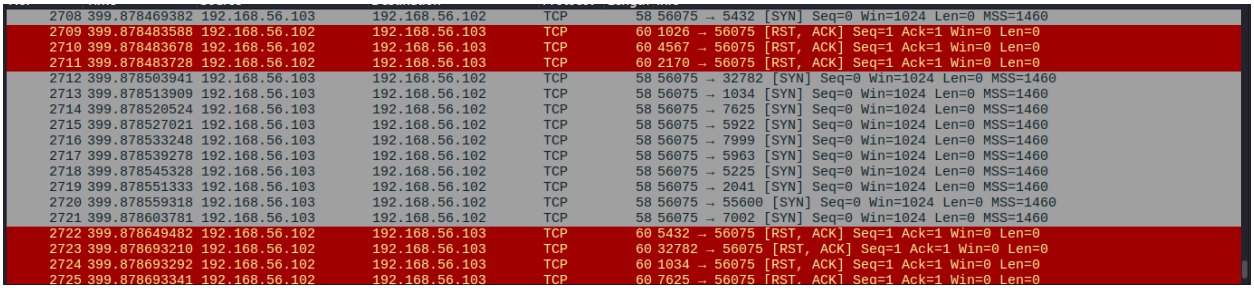
56075 → 9000 [SYN]

56075 → 3784 [SYN]

56075 → 4848 [SYN]

This shows that everything is closed

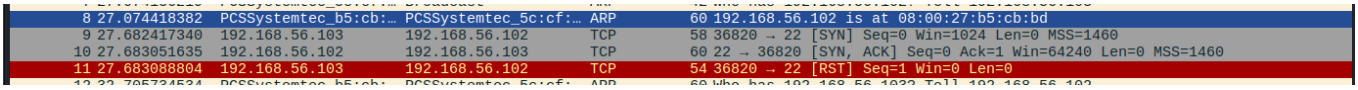
[RST] = REST, NO SERVICE HERE



Finding open ports

```
Sudo nmap -p 22 192.168.56.102
```

You can see the 3-way handshake.= open port

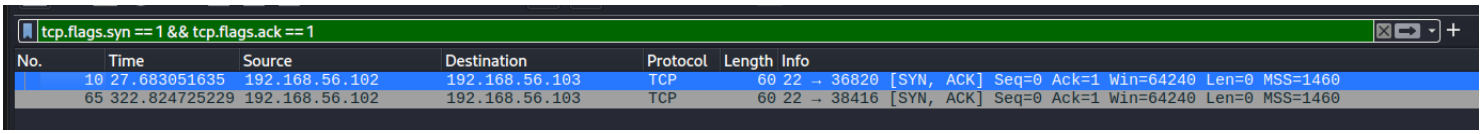


Finding open ports through a full scan by filter

`Nmap -sS 192.168.56.102`

Apply this **filter**

`tcp.flags.syn == 1 && tcp.flags.ack == 1`

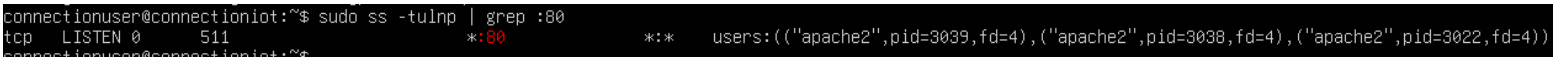


Opening more ports to the public for the purpose of this lab only.
On Ubuntu machine

1. HTTP (PORT 80)

Sudo apt update
Sudo spt install apache2 -y

VERIFY
Sudo ss -tulnp | grep :80



On wireshark filter (http)

- GET requests
- HTTP responses
- Headers

2. HTTPS (port 443)

Sudo apt install openssl -y (install SSL support)
We can see on wireshark filter (tls)

- TLS handshake
- Certificates
- Encrypted web traffic

3. DNS (port 53)

Sudo apt install bind9 -y (install a dns server)
On wireshark filter (dns)

- Queries
- Reponses
- Domain lookups

4. SMB (ports 445/139)

Sudo apt install samba -y (install samba)

On wireshark filter (smb2)

- File sharing
- Authentication
- Windows traffic

5. FTP (port 21)

Sudo apt install vsftpd -y

On wireshark filter (ftp)

Username and passwords in plaintext.

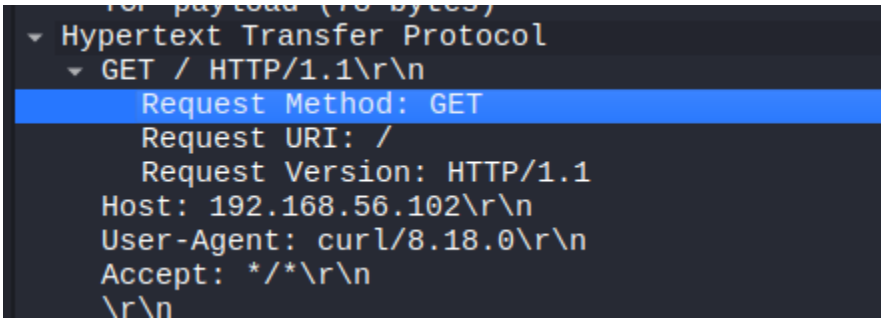
FTP is not a secure protocol.

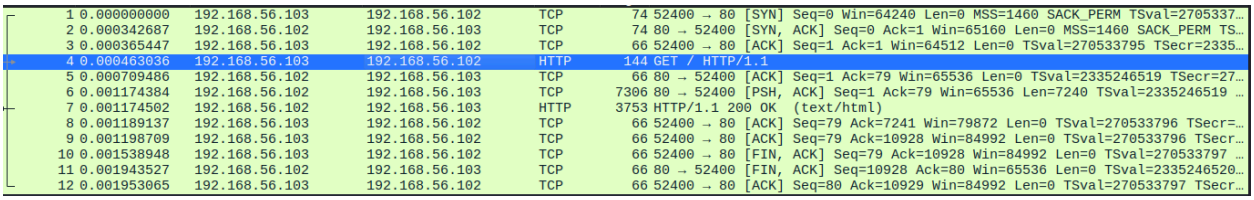
HTTP

I searched up the IP address and everything on wireshark displays the type of {request method}, {the host}, {user-agent}: this is not a secure protocol

Results show

- TLS Handshake,
- Certificate exchange
- Source IP
- Destination IP
- Encrypted application data.

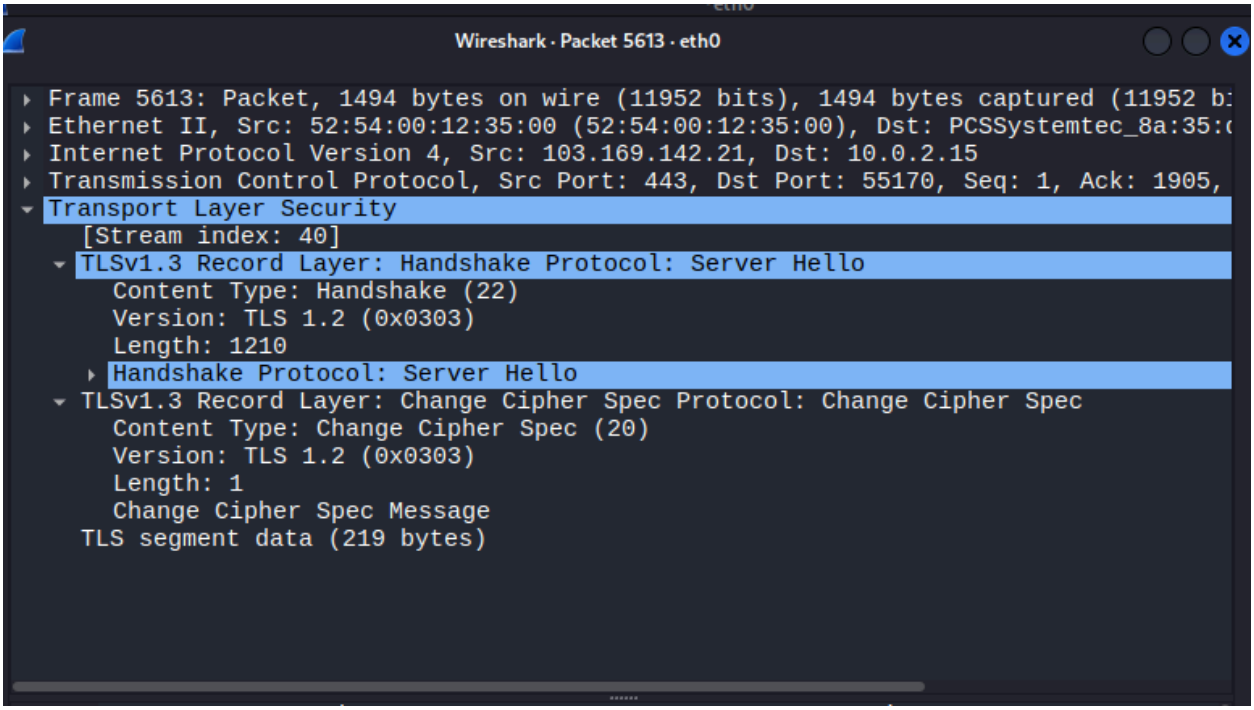




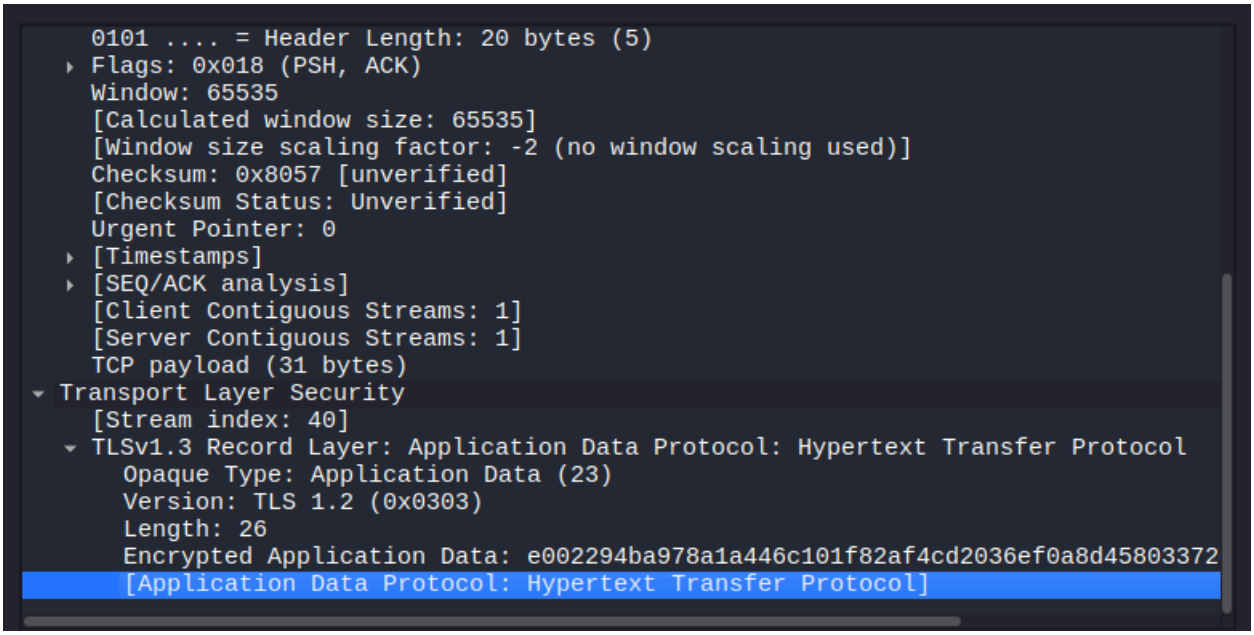
HTTPS

Unlike HTTP, the content of the communication are encrypted. Wireshark can still show information such as source and destination IP address, TLS handshake, certificate, and packet length, but cannot display the actual web content, username, or password, and data being exchanged.

HTTPS using TLS

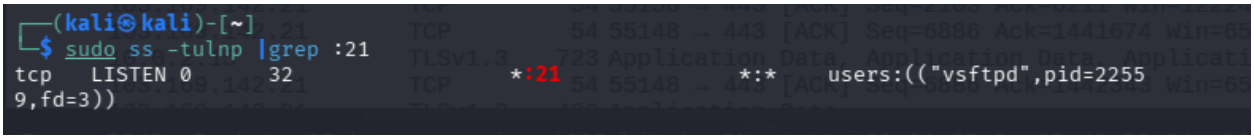


- TLS client Hello
- TLS server hello
- Certificate exchange
- Encrypted application data



FTP (unsecure protocol)

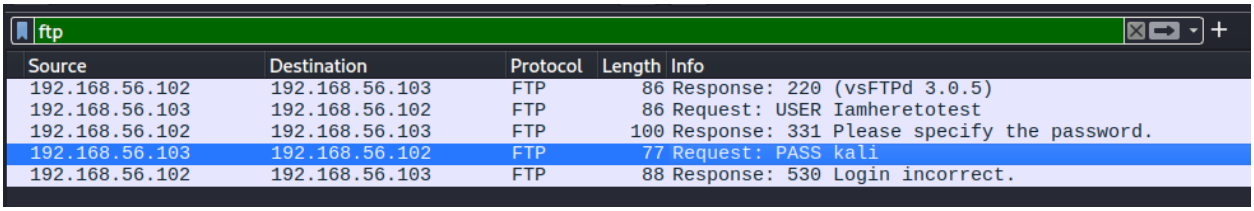
Run FTP on machine



Generate FTP traffic on KALI

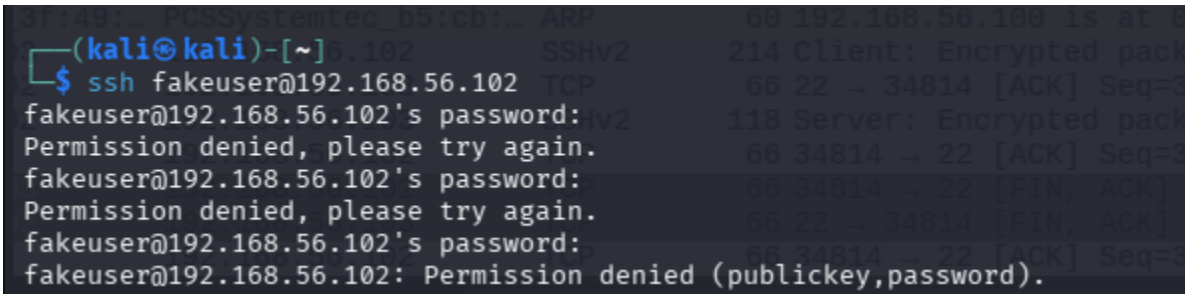
```
Ftp 192.168.56.102
```

Every information inputted appeared on Wireshark no encryption, clear text



Failed SSH Login Investigation

Generate authentication failure —- then investigate



Verify failure on the machine

`Journalctl -n 50`

```
connectioniot vsftpd[2155]: pam_unix(vsftpd:auth): check pass; user unknown
connectioniot vsftpd[2155]: pam_unix(vsftpd:auth): authentication failure; logname= uid=0 euid=0 tty=ftp ruser=Iamhe
connectioniot systemd[1]: Starting sysstat-collect.service - system activity accounting tool...
connectioniot systemd[1]: sysstat-collect.service: Deactivated successfully.
connectioniot systemd[1]: Finished sysstat-collect.service - system activity accounting tool.
connectioniot kernel: workqueue: drm_fb_helper_damage_work hogged CPU for >10000us 4 times, consider switching to WQ
connectioniot sshd-session[2165]: Invalid user fakeuser from 192.168.56.103 port 34814
connectioniot sshd-session[2165]: pam_unix(sshd:auth): check pass; user unknown
connectioniot sshd-session[2165]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser=
connectioniot sshd-session[2165]: Failed password for invalid user fakeuser from 192.168.56.103 port 34814 ssh2
connectioniot sshd-session[2165]: pam_unix(sshd:auth): check pass; user unknown
connectioniot sshd-session[2165]: Failed password for invalid user fakeuser from 192.168.56.103 port 34814 ssh2
connectioniot sshd-session[2165]: pam_unix(sshd:auth): check pass; user unknown
connectioniot sshd-session[2165]: Failed password for invalid user fakeuser from 192.168.56.103 port 34814 ssh2
connectioniot sshd-session[2165]: Connection closed by invalid user fakeuser 192.168.56.103 port 34814 [preauth]
```

Check for failed passwords

`Journalctl | grep “Failed password”`

```
connectionuser@connectioniot:~$ journalctl | grep "Failed password"
Jun 01 17:46:21 connectioniot sudo[2228]: connectionuser : TTY=/dev/tty1 ; PWD=/home/connectionuser ; USER=root ; COMMAND=/usr/bin/grep Failed passwo
Jun 01 17:47:40 connectioniot sudo[2259]: connectionuser : TTY=/dev/tty1 ; PWD=/home/connectionuser ; USER=root ; COMMAND=/usr/bin/grep Failed passwo
Jun 02 03:04:44 connectioniot sshd-session[2165]: Failed password for invalid user fakeuser from 192.168.56.103 port 34814 ssh2
Jun 02 03:04:52 connectioniot sshd-session[2165]: Failed password for invalid user fakeuser from 192.168.56.103 port 34814 ssh2
Jun 02 03:04:57 connectioniot sshd-session[2165]: Failed password for invalid user fakeuser from 192.168.56.103 port 34814 ssh2
```

Results

A failed SSH login event was intentionally generated from the Kali Linux VM to the Ubuntu server. Multiple authentication attempts were made using an invalid username and password. The event was investigated using `journalctl` and SSH authentication logs. The source IP, targeted username, and failure status were identified. This demonstrated how security analysts investigate authentication failure alerts and identify potential brute-force activity.

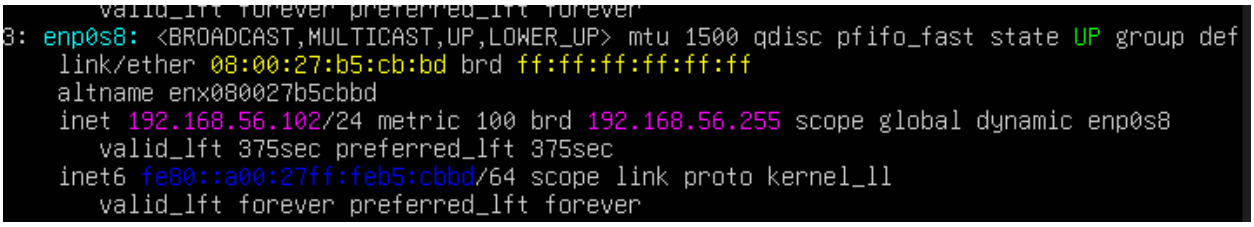
ATTACK— ARP Spoofing (Man in the middle)

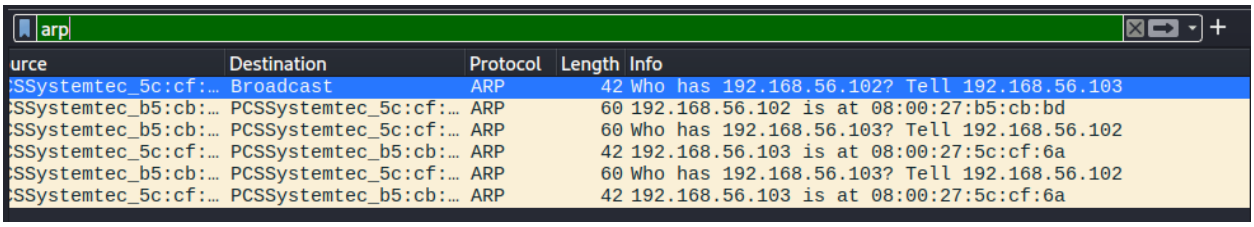
Current VMs
-Kali IP: 192.168.56.103
-Ubuntu IP: 192.168.56.102
-windows 11: 192.168.56.101

Install tool on kali

```
Sudo apt update
Sudo apt install dsniff -y
```

Wireshark filter arp
Generate normal arp ping 192.168.56.102 on kali.





The address resolution protocol was captured in wireshark to observe how devices resolve IP addresses to MAC addresses on the local network.

- ARP Request: “who has 192.168.65.102?”
- ARP Reply: “192.168.56.102 is at 08:00:27:b5:cb:bd”
- Communication is normal now with no interference.

Enable packet forwarding on KALI

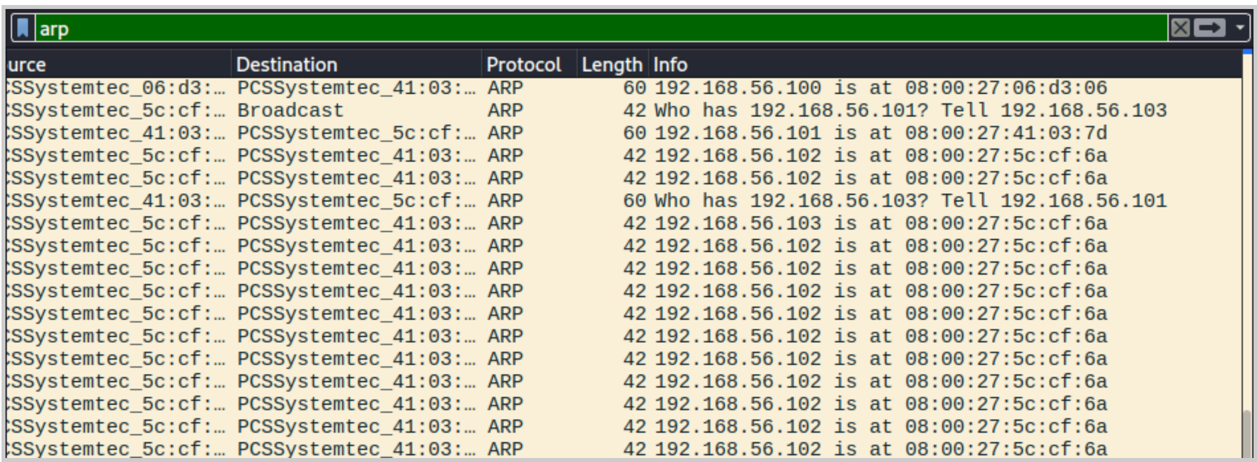
```
sudo sysctl -w net.ipv4.ip_forward=1
```

Start ARP Spoofing

On Terminal #1

```
sudo arpspoof -i eth1 -t 192.168.56.101 192.168.56.102
```

- Tell windows that Kali is Ubuntu



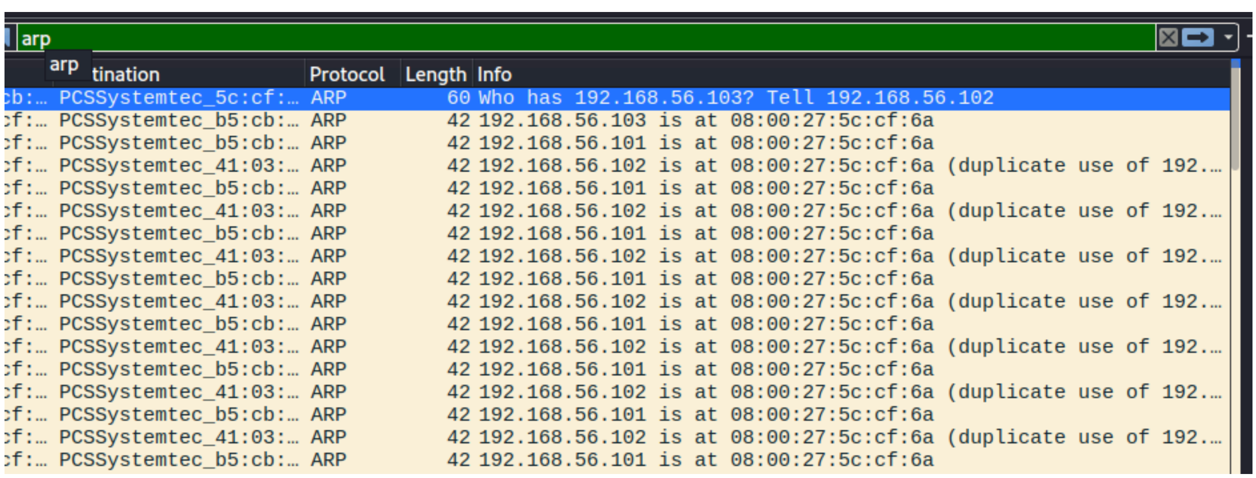
arp

urce	Destination	Protocol	Length	Info
:SSystemtec_06:d3:…	PCSSystemtec_41:03:…	ARP	60	192.168.56.100 is at 08:00:27:06:d3:06
:SSystemtec_5c:cf:…	Broadcast	ARP	42	Who has 192.168.56.101? Tell 192.168.56.103
:SSystemtec_41:03:…	PCSSystemtec_5c:cf:…	ARP	60	192.168.56.101 is at 08:00:27:41:03:7d
:SSystemtec_5c:cf:…	PCSSystemtec_41:03:…	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a
:SSystemtec_5c:cf:…	PCSSystemtec_41:03:…	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a
:SSystemtec_41:03:…	PCSSystemtec_5c:cf:…	ARP	60	Who has 192.168.56.103? Tell 192.168.56.101
:SSystemtec_5c:cf:…	PCSSystemtec_41:03:…	ARP	42	192.168.56.103 is at 08:00:27:5c:cf:6a
:SSystemtec_5c:cf:…	PCSSystemtec_41:03:…	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a
:SSystemtec_5c:cf:…	PCSSystemtec_41:03:…	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a
:SSystemtec_5c:cf:…	PCSSystemtec_41:03:…	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a
:SSystemtec_5c:cf:…	PCSSystemtec_41:03:…	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a
:SSystemtec_5c:cf:…	PCSSystemtec_41:03:…	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a
:SSystemtec_5c:cf:…	PCSSystemtec_41:03:…	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a
:SSystemtec_5c:cf:…	PCSSystemtec_41:03:…	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a
:SSystemtec_5c:cf:…	PCSSystemtec_41:03:…	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a
:SSystemtec_5c:cf:…	PCSSystemtec_41:03:…	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a
:SSystemtec_5c:cf:…	PCSSystemtec_41:03:…	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a

Terminal #2

```
sudo arpspoof -i eth1 -t 192.168.56.102 192.168.56.101
```

- Tell Ubuntu that Kali is Windows

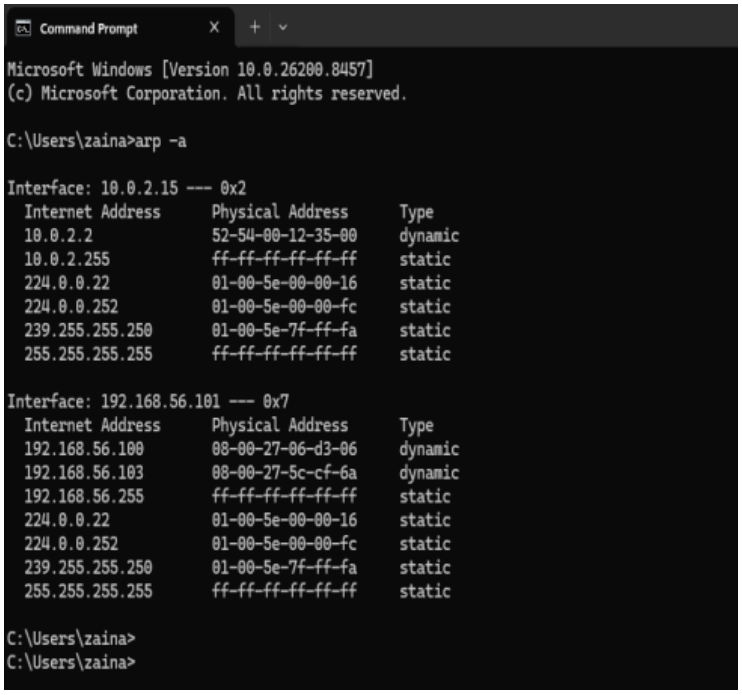


arp

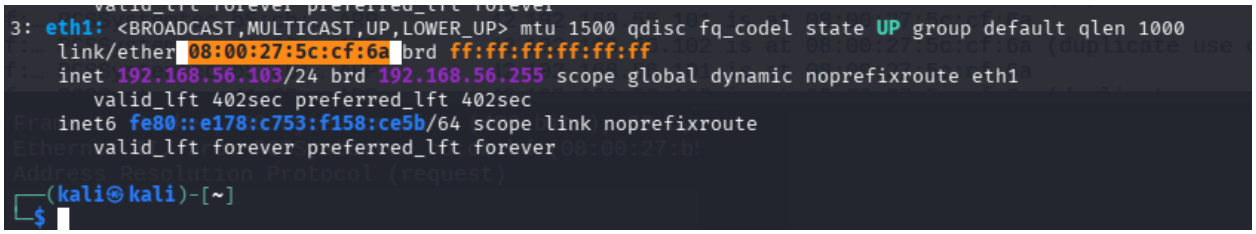
	tination	Protocol	Length	Info
cb:…	PCSSystemtec_5c:cf:…	ARP	60	Who has 192.168.56.103? Tell 192.168.56.102
:f:…	PCSSystemtec_b5:cb:…	ARP	42	192.168.56.103 is at 08:00:27:5c:cf:6a
:f:…	PCSSystemtec_b5:cb:…	ARP	42	192.168.56.101 is at 08:00:27:5c:cf:6a
:f:…	PCSSystemtec_41:03:…	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a (duplicate use of 192.…
:f:…	PCSSystemtec_b5:cb:…	ARP	42	192.168.56.101 is at 08:00:27:5c:cf:6a
:f:…	PCSSystemtec_41:03:…	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a (duplicate use of 192.…
:f:…	PCSSystemtec_b5:cb:…	ARP	42	192.168.56.101 is at 08:00:27:5c:cf:6a
:f:…	PCSSystemtec_41:03:…	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a (duplicate use of 192.…
:f:…	PCSSystemtec_b5:cb:…	ARP	42	192.168.56.101 is at 08:00:27:5c:cf:6a
:f:…	PCSSystemtec_41:03:…	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a (duplicate use of 192.…
:f:…	PCSSystemtec_b5:cb:…	ARP	42	192.168.56.101 is at 08:00:27:5c:cf:6a
:f:…	PCSSystemtec_41:03:…	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a (duplicate use of 192.…
:f:…	PCSSystemtec_b5:cb:…	ARP	42	192.168.56.101 is at 08:00:27:5c:cf:6a
:f:…	PCSSystemtec_41:03:…	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a (duplicate use of 192.…
:f:…	PCSSystemtec_b5:cb:…	ARP	42	192.168.56.101 is at 08:00:27:5c:cf:6a
:f:…	PCSSystemtec_41:03:…	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a (duplicate use of 192.…
:f:…	PCSSystemtec_b5:cb:…	ARP	42	192.168.56.101 is at 08:00:27:5c:cf:6a

Observation

- Multiple replies claiming ownership indicated ARP poisoning, for both Terminals.
- Duplicate IP warning (wireshark detects conflicting ARP information and flags it as duplicate).



Checking ARP table on windows 11 shows that the it was changed to Kali's MAC address



Detection summary:
An ARP spoofing attack was simulated in this slab using Kali Linux. The attacker system sent forged ARP replies to both the Windows and Ubuntu Hosts, attempting to associate the attacker's MAC address with legitimate addresses.

Findings

- Repeated ARP replies were observed in Wireshark.
- ARP cache entries on the victim system (windows 11) changed during the attack)
- The attacker attempted to position itself between the two communicating hosts (Ubuntu and windows 11).

Indicators of compromise:

- Multiple ARP replies without corresponding requests.
- IP addresses resolving to unexpected MAC addresses.
- Frequent ARP updates in a short time period.

Last DEMO showing ARP table before and right after spoofing attack
WINDOWS 11
BEFORE

```
C:\Users\zaina>arp -a

Interface: 10.0.2.15 --- 0x2
  Internet Address      Physical Address      Type
  10.0.2.2              52-54-00-12-35-00     dynamic
  10.0.2.255            ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static

Interface: 192.168.56.101 --- 0x7
  Internet Address      Physical Address      Type
  192.168.56.100        08-00-27-06-d3-06     dynamic
  192.168.56.102        08-00-27-b5-cb-bd     dynamic
  192.168.56.103        08-00-27-5c-cf-6a     dynamic
  192.168.56.255        ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static
```

AFTER

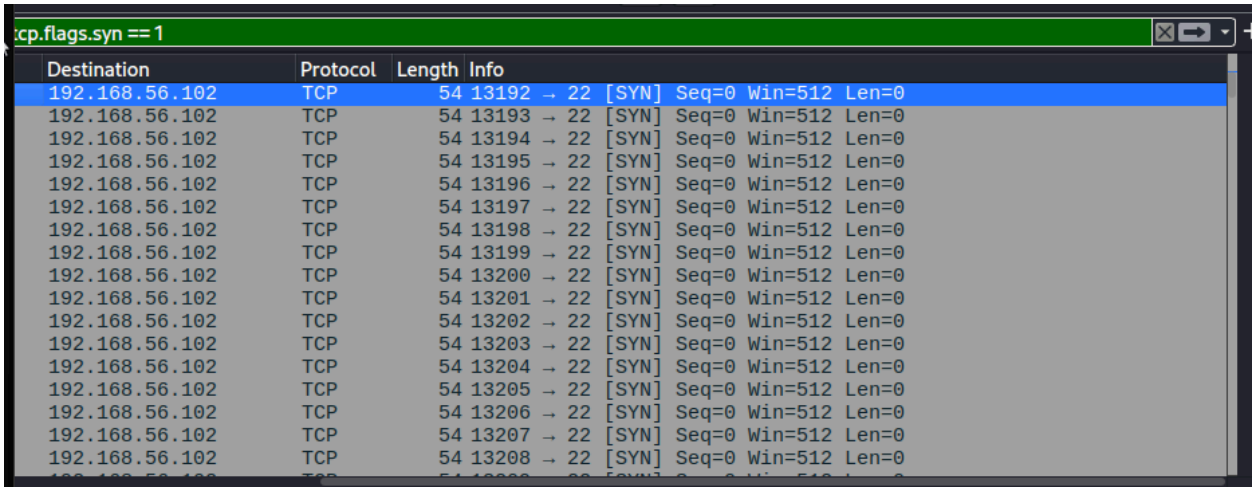
```
C:\Users\zaina>arp -a

Interface: 10.0.2.15 --- 0x2
  Internet Address      Physical Address      Type
  10.0.2.2              52-54-00-12-35-00     dynamic
  10.0.2.255            ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static

Interface: 192.168.56.101 --- 0x7
  Internet Address      Physical Address      Type
  192.168.56.100        08-00-27-06-d3-06     dynamic
  192.168.56.102        08-00-27-5c-cf-6a     dynamic
  192.168.56.103        08-00-27-5c-cf-6a     dynamic
  192.168.56.255        ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static
```

ATTACK — SYNC flood detection

TCP normal connection looks like
`SYN`
`SYN-ACK`
`ACK`

Wireshark filter tcp.flags.syn == 1

Kali Linux install
`Sudo hping3 -S -p 22 -flood 192.168.56.102` (to generate flood)
-S = SYN flag
-p 22 = target SSH
–flood = send rapidly



Documentation:
A SYN flood attack was simulated in the lab environment using hping3. The attack generated a large number of TCP SYN packets toward the SSH service running on the Ubuntu server. Wireshark was used to monitor the traffic and identify abnormal connection behavior.

Observations:

- Thousands of TCP SYN packets were observed causing my whole windows 11 to become slow. It is also a compromise of a DDOS attack.

- Connections were initiated but not completed (SYN, SYN-ACK, ACK)
- Excessive connection attempts occurred within a short time period.

Indicators of compromise

- Large volume of SYN packets from a single source
- Repeated connection attempts without successful session establishment
- Resource exhaustion on the target system.

ATTACK — DNS Spoofing/ DNS Poisoning detection

This will explain

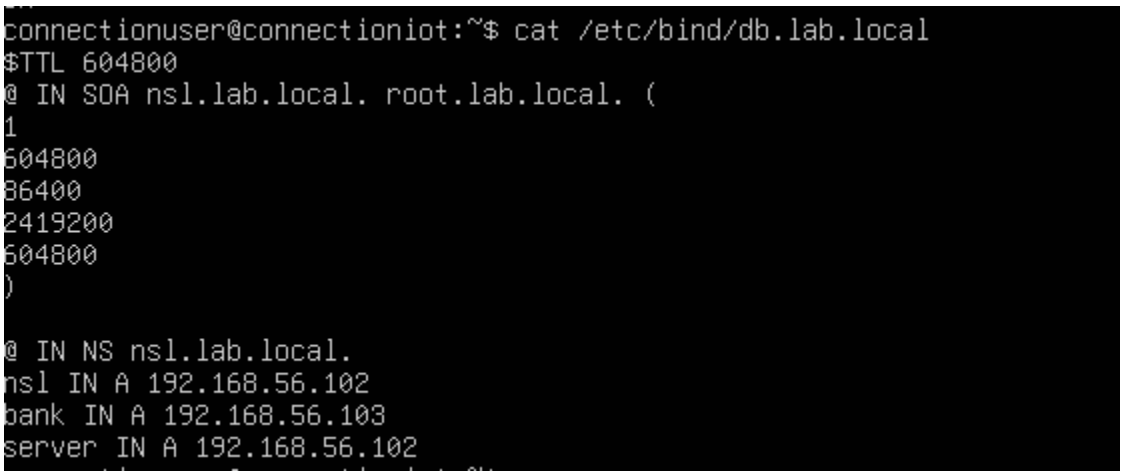
- Fake websites
- Phishing
- Malware redirection
- DNS hijacking

SAFE DNS poisoning Simulation

Verifying that Ubuntu server is running a DNS service

```
Sudo ss -tulnp | grep :53
```

1. Create my own fake domain
   Create a zone file
   Called: **lab.local**
   ```
   Sudo nano /etc/bind/db.lab.local
   ```



Create Configuration File

`sudo nano /etc/bind/named.conf.local`

```
connectionuser@connectioniot:~$ cat /etc/bind/named.conf.local
zone "lab.local" {
type master;
file "/etc/bind/db.lab.local";
};
```

Verify files

```
connectionuser@connectioniot:~$ sudo named-checkzone lab.local /etc/bind/db.lab.local
zone lab.local/IN: loaded serial 1
OK
```

Restart and check

```
connectionuser@connectioniot:~$ sudo systemctl restart bind9
connectionuser@connectioniot:~$ nslookup bank.lab.local 192.168.56.102
Server:         192.168.56.102
Address:        192.168.56.102#53

Name:   bank.lab.local
Address: 192.168.56.103
```

The goal of this lab was to understand how DNS records can influence where users are directed when they visit a website. This demonstrates the core concept behind DNS poisoning attacks, where an attacker manipulates DNS responses to redirect users to unintended systems.

Instead of targeting real websites, a controlled lab environment was used to safely simulate the behavior.

What Normal looks like

User

↓

DNS Query

↓

Legitimate Website

Poisoned response looks like

User

↓

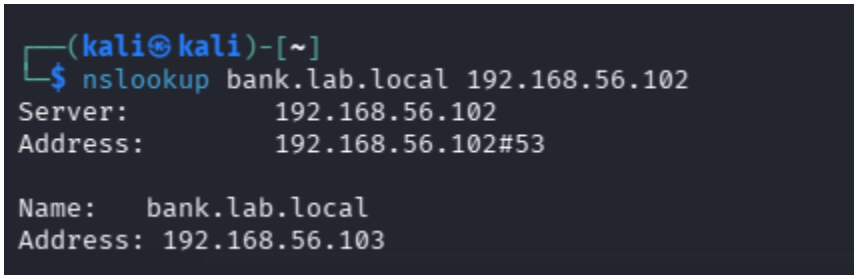
DNS Query

↓

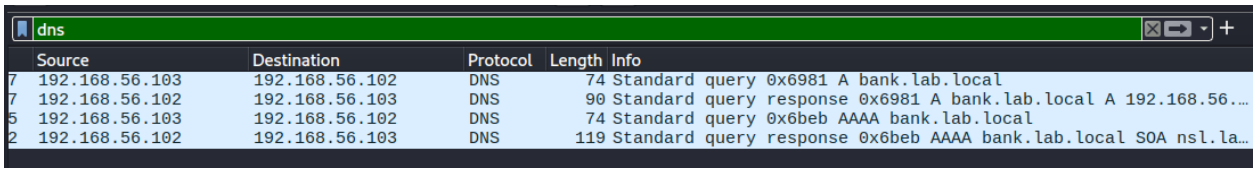
Attacker-Controlled System

NOW THAT WE created a domain lets query from Kali the process is the victims would think that they are accessing a bank server but really attackers tricks DNS and instead the attacker server not bank server.

Query on Kali



Wireshark captures



Basic summary

Whenever someone asks for bank.lab.local, send them to Kali.

if Kali was running a fake banking website, the victim would end up on Kali instead of the real server.

Note it would look like a legit website just like a banking website to trick the victim.

Indicators of compromise:

- DNS response contains unexpected IP addresses. (not matching theirs)
- internal/private IPs returned for public websites.
- Multiple conflicting DNS responses for the same domain
- Immediate follow of suspicious DNS responses.